LAB 10 – CODE REVIEW AND QUALITY

Using AI to Improve Code Quality and Readability

Name: D.POORNANDA CHALAM **Roll No:** 2503A52361 **Batch:** 13

Task 1 – Syntax and Logic Errors

Question / Objective:
Use AI to identify and fix syntax and logic errors in the given Python script that calculates a student's average score.

AI Prompt:
"Here is a Python function I wrote to compute average test scores, but it keeps throwing syntax errors and might crash on empty input lists. Could you help me debug the syntax issues, fix the misspelled variable, and add a safety check for empty data?"

Source Code:

```
def calc_average(marks):
    """Calculate the average score safely with empty list handling."""
    if not marks:
        return 0
    total = 0
    for mark in marks:
        total += mark
    average = total / len(marks)
    return average

marks = [85, 90, 78, 92]
print("Average Score is", calc_average(marks))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab10\task1.py"
Average Score is 86.25

[Done] exited with code=0 in 0.154 seconds
```

Justification & Technical Explanation:
The original snippet suffered from a missing colon after the for-loop statement and an unclosed parenthesis on the final print function call. The variable 'avrage' was also misspelled during assignment. Beyond correcting these syntax bugs, I added an early return guard (`if not marks: return 0`) to prevent an unhandled ZeroDivisionError when an empty score collection is supplied. With input values [85, 90, 78, 92], the function computes the correct average of 86.25.

Task 2 – PEP 8 Compliance

Question / Objective:
Refactor the rectangle-area program to follow PEP 8 style guidelines.

AI Prompt:
"Rewrite this Python code using PEP 8 naming, spacing, and formatting conventions. Ensure standard snake_case naming, clean spacing around operators, a descriptive docstring, and multi-line readability while keeping the arithmetic logic intact."

Source Code:

```
def area_of_rectangle(length, breadth):
    """Return the area of a rectangle given length and breadth."""
    return length * breadth


print(area_of_rectangle(10, 20))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab10\task2.py"
200

[Done] exited with code=0 in 0.132 seconds
```

Justification & Technical Explanation:
Refactored the script to align fully with PEP 8 standards: converted the function and parameter identifiers to lowercase snake_case (`area_of_rectangle`, `length`, `breadth`), placed spaces around binary operators, separated top-level module statements with two blank lines, and added a concise triple-quoted docstring explaining the operation. The calculation produces the exact expected geometric area of 200.

Task 3 – Readability Enhancement

Question / Objective:
Improve readability without changing the logic. Use descriptive variable names, inline comments, and clear formatting.

AI Prompt:
"Improve the readability of this percentage calculation script without modifying its underlying formula. Use meaningful variable names, clear formatting, and a helpful inline comment."

Source Code:

```
def calculate_percentage(value, percentage):
    """Calculate a percentage of a given value."""
    # Multiply the value by the percentage and divide by 100.
    return value * percentage / 100


total_value = 200
percentage_rate = 15
result = calculate_percentage(total_value, percentage_rate)
print(result)
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab10\task3.py"
30.0

[Done] exited with code=0 in 0.141 seconds
```

Justification & Technical Explanation:
Replaced vague single-letter names like 'c' with self-documenting identifiers `calculate_percentage`, `total_value`, and `percentage_rate`. An inline comment documents the arithmetic formula `(value * percentage) / 100`, making the function easily maintainable across a shared engineering codebase while producing the correct floating-point answer of 30.0.

Task 4 – Refactoring for Maintainability

Question / Objective:
Refactor repetitive welcome messages into a reusable function.

AI Prompt:
"Refactor the repeated welcome print statements into a reusable function and loop through the student list. Keep the same messages."

Source Code:

```
def welcome_student(student_name):
    """Print a welcome message for one student."""
    print("Welcome", student_name)


students = ["Alice", "Bob", "Charlie"]
for student in students:
    welcome_student(student)
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab10\task4.py"
Welcome Alice
Welcome Bob
Welcome Charlie

[Done] exited with code=0 in 0.138 seconds
```

Justification & Technical Explanation:
Hardcoded sequential print statements violate the DRY (Don't Repeat Yourself) principle. By abstracting the greeting into `welcome_student()` and driving it through an iterable list, the code becomes modular, extensible, and clean. New students can be supported simply by updating the data structure without touching display logic.

Task 5 – Performance Optimization

Question / Objective:
Optimize the program that calculates squares of numbers from 1 to 999,999.

AI Prompt:
"Optimize this Python code for speed and simplicity. Avoid creating an unnecessary intermediate list of numbers, preserve the final count, and explain the change."

Source Code:

```
# Generate squares directly using a list comprehension.
squares = [number ** 2 for number in range(1, 1_000_000)]
print(len(squares))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab10\task5.py"
999999

[Done] exited with code=0 in 0.362 seconds
```

Justification & Technical Explanation:
The legacy implementation created an unnecessary intermediate list of raw numbers and invoked `append()` in an interpreted for-loop for 1,000,000 cycles. Replacing it with a list comprehension executes in optimized C-level Python bytecode, significantly reducing CPU instruction overhead and memory churn while verifying all 999,999 squared elements are created.

Task 6 – Complexity Reduction

Question / Objective:
Simplify the nested if-else logic used to assign a grade based on a score.

AI Prompt:
"Simplify the nested grade function using elif. Preserve the existing grade boundaries and return values, and include a few test cases."

Source Code:

```
def grade(score):
    """Return a letter grade based on the score."""
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"


print(grade(95))
print(grade(82))
```

```
print(grade(74))
print(grade(65))
print(grade(40))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab10\task6.py"
A
B
C
D
F

[Done] exited with code=0 in 0.129 seconds
```

Justification & Technical Explanation:

Deeply nested conditional branches increase cognitive complexity and nesting depth. Refactoring into an ordered `if-elif-else` evaluation ladder flattens the structure. Because conditions are ordered from highest score threshold to lowest, the first passing condition returns immediately, yielding cleaner code with identical grading logic.